Tuwaiq CLUB

أكاديمية طويق
Tuwaiq Academy

KFUPM
Student Affairs

DAY 4

Security & configuration

Token authentication, ownership, and deployment readiness

Wednesday, October 28, 2026

Your five-day journey

Yesterday: validated data. Today: private records and environment configuration.

01

Foundation

Project, Task model, migrations, admin

DONE

02

Core API

CRUD with function views

DONE

03

Serialization & validation

Serializers, rules, clear field errors

DONE

04

Security & configuration

Tokens, ownership, environment

TODAY

05

Publish

Deploy, verify, README, live demo

Today's session

Learn the security boundary, check it with two users, and prepare configuration

Explain + live demo
~60 min

Exercise
~60 min

Break
short

Project
~60 min

Exit check
end of day

Priority rule: finish the exercise core, then move to the project. Stretch work can wait.

Learning outcomes

By the end of today you can...

01 Secure requests with token authentication

02 Filter each user's data in every function view

03 Use one owner-aware helper for retrieve, update and delete

04 Assign ownership on the server; keep owner read-only

05 Move configuration into the environment and prepare packages

Authentication and authorization

Security starts with identity, then checks access

Authentication · who is calling?
A token proves the caller's identity

Authorization · what may they do?
Ownership limits the caller's data

401
no or invalid token

403
permission is denied

404
another user's object is invisible

In this program, another user's task returns 404, not 403.

Token authentication

A login request exchanges credentials for a key used on later requests

Username + password → **POST /api/auth/token/** → **Token key** → **Authorization: Token <key>**

Token first

List TokenAuthentication before SessionAuthentication so missing credentials return 401.

A token acts like a password. Never put it in Git, screenshots, or shared evidence.

Postman request setup

Keep the server address and token in an environment

Environment

`base_url = http://127.0.0.1:8000`

token = empty until login

Login request

POST `{{base_url}}/api/auth/token/`

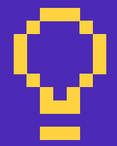
Save the returned token

Protected request

`Authorization: Token {{token}}`

On every API request

```
// post-response script on the login request
const data = pm.response.json();
pm.environment.set("token", data.token);
```



LIVE DEMO

Attack and fix

20 min: Bob reaches Alice's task, then we close the hole

Before	Fix lists	Fix objects	After
unfiltered lookup	filter by owner	get_own_task()	Bob gets 404

Creation already uses save(owner=request.user); a client-supplied owner is ignored.

Ownership is a query boundary

The safest object permission starts before the object is found

Alice's queryset
Alice's tasks only

Bob's queryset
Bob's tasks only

GET /api/tasks/{alice_id}/

Bob searches Bob's queryset

Alice's object is absent

404

404 hides that another user's data even exists.

One ownership helper

Use the same lookup in get_task, update_task and delete_task

```
def get_own_task(request, pk):
    return get_object_or_404(Task, pk=pk, owner=request.user)

@api_view(["GET"])
def get_task(request, pk):
    task = get_own_task(request, pk)
    return Response(TaskSerializer(task).data)
```

For lists: Task.objects.filter(owner=request.user). Filter before returning data.

Defense in depth

Three required layers protect identity, access and ownership

1. IsAuthenticated

Reject anonymous requests before the view runs: 401

2. Owner filter

List: filter by request.user
Objects: get_own_task()

3. Owner on save

save(owner=request.user)
owner stays read-only

Another user's object is absent from the allowed query: return 404.

Configuration belongs in the environment

The code reads runtime values instead of storing secrets in the repository

Variable	Type	Purpose
`SECRET_KEY`	string	private runtime value
`DEBUG`	boolean	local or production mode
`ALLOWED_HOSTS`	list	approved host names

```
SECRET_KEY = os.getenv("SECRET_KEY", "development-only-key")
DEBUG = os.getenv("DEBUG", "True").lower() == "true"
```

.env.example holds placeholders. The real .env stays ignored and never enters Git.

Deployment packages

The dependency file tells the production platform what to install

psycopg[binary]
PostgreSQL driver

dj-database-url
reads DATABASE_URL

gunicorn
production server

whitenoise
static files

requirements.txt → **pip install** → **pip check**

Your laptop and the deployment platform install the same declared versions.

HOUR 2 · EXERCISE

Personal Reading List

Alice and Bob each see and change only their own items

Core	Time	Stretch
Tasks 1-2	about 45 min	Task 3

Start in days/day-04/exercise/. Use Postman to record the expected responses.

Exercise Task 1: token authentication

config/settings.py and config/urls.py

01 Add rest_framework.authtoken, then run migrate

02 Configure TokenAuthentication before SessionAuthentication

03 Add the login endpoint: POST /api/auth/token/

Request	Expected
Valid credentials	200 + token
Wrong password	400
Reading list with no token	401
Books with Alice's token	200 + 6 books

Exercise Task 2: ownership

catalog/views.py; the existing get/update/delete views all use the helper

Task	Change	Result
2a	list_items: filter by request.user	Alice sees 1, 2; Bob sees 3
2b	get_own_item: id + user	Bob cannot read or change Alice's item
2c	create_item: save(user=request.user)	Body cannot choose the owner

Task 3 stretch: use serializer context for the current user.
Reject a past target_date or a book already on that user's list.

BREAK

Short break

After the break: secure your own Task Management API

HOUR 3 · PROJECT

Secure and configure the Task API

Finish security and its two-user check before configuration

Commit Day 3	Checkpoint	P1-P5	P6-P8
save your work	day-04	security: ~40 min	config: ~15 min

Keep Git history, .venv and database. The README template is completed on Day 5.

Day 4 project: Part 1, security

P1-P5; complete the two-user check before Part 2

TODO	File	What you build
P1	`config/settings.py`	Add authtoken, then migrate
P2	`config/settings.py`	Token authentication before Session
P3	`config/urls.py`	obtain_auth_token; api/auth/token/
P4	`tasks/views.py`	list_tasks: only the caller's tasks
P5	`tasks/views.py`	get_own_task: look up by id AND owner

get_task, update_task and delete_task all call get_own_task().

Day 4 project: Part 2, configuration

P6-P8; prepare the same code to run in a different environment

TODO	File	What you build
P6	`config/settings.py`	SECRET_KEY, DEBUG, ALLOWED_HOSTS from environment variables
P7	`.env.example`	Document variables with placeholders only
P8	`requirements.txt`	psycopg[binary], dj-database-url, gunicorn, whitenoise; install -r

```
python manage.py runserver  # works with development defaults
python -m pip check         # No broken requirements found.
git status                  # .env must not be listed
```

The two-user security check

Record results without tokens; each user must own at least one task

#	Request	Expect
1	POST /api/auth/token/ (Alice)	200 + token
2	POST /api/auth/token/ (wrong password)	400
3	GET /api/tasks/ (no token)	401
4	GET /api/tasks/ (Alice)	200; only Alice's tasks
5	POST /api/tasks/create/ (body owner = Bob)	201; owner = Alice
6	GET /api/tasks/{alice_id}/ (Bob)	404
7	PATCH /api/tasks/{alice_id}/update/ (Bob)	404; unchanged
8	DELETE /api/tasks/{alice_id}/delete/ (Bob)	404; still exists

Day 4 exit check

Ready for the next day when every check passes

01 Valid login returns a token; no token on protected routes -> 401

02 Creation ignores a client-supplied owner

03 Each user's list contains only their own records

04 Cross-user retrieve, update and delete -> 404; nothing changes

05 Environment settings, placeholder .env.example, clean pip check

06 The two-user check is recorded; the commit is pushed

Tomorrow the same code goes live

Optional homework for Day 5

About 15 minutes. The README template you got today is completed on Day 5.

01 Open your Neon and Render accounts (part of the readiness check)

02 Neon: create the project tuwaiq-django-api and find the connection string

03 Render: New → Web Service, check you can see task-management-api, then stop

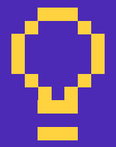
04 Read setup/deployment.md once, end to end

Do not paste the connection string anywhere yet.

Tuwaiq CLUB

أكاديمية طويق
Tuwaiq Academy

KFUPM
Student Affairs



Day 4 complete

The Task API is secured and configurable, ready to move to production.

Next: deployment, smoke checks, README, and a live demonstration